FortuneCloud
We Understand Technology

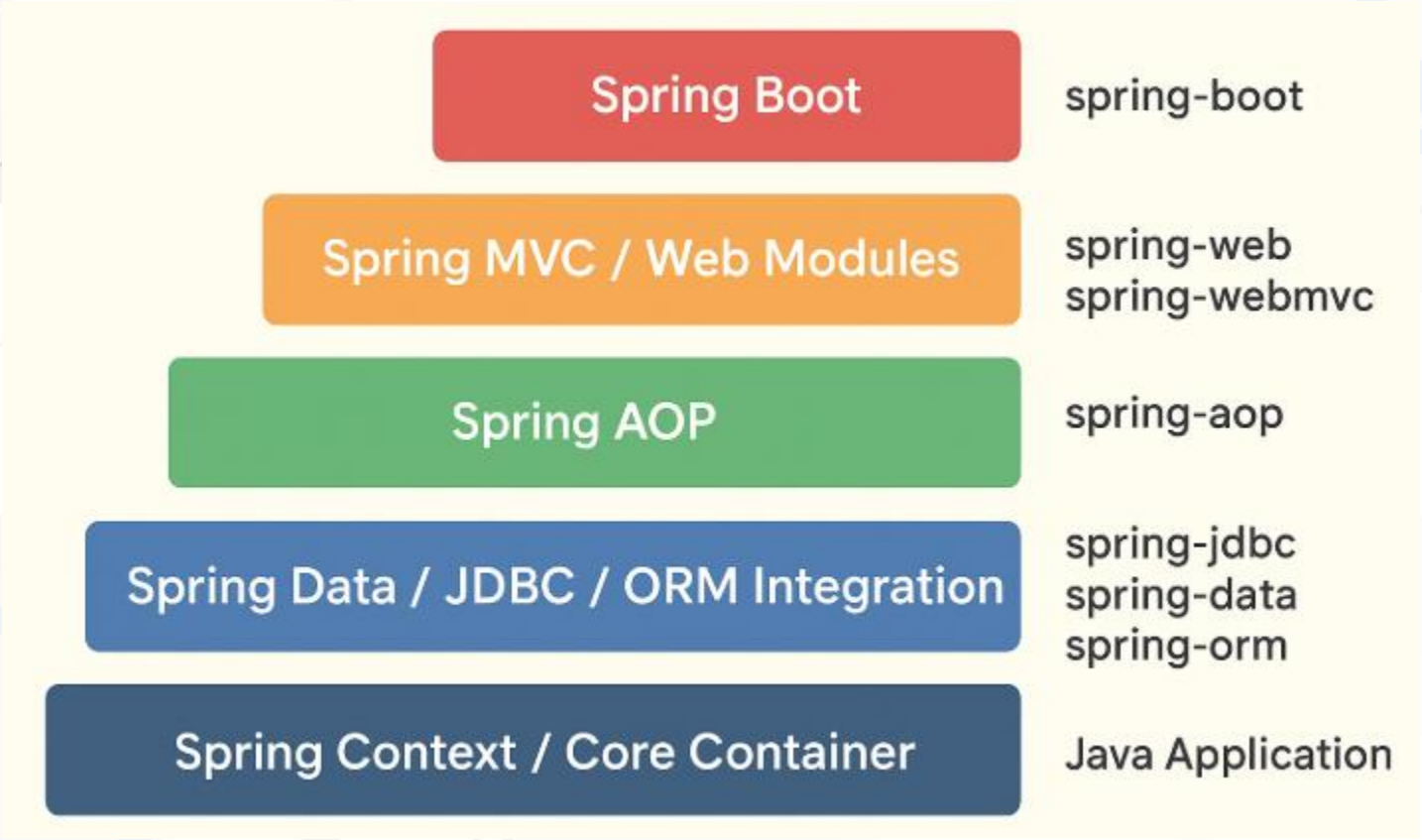
WELCOME

Introduction

- Spring Framework is a **powerful and lightweight** Java framework used to build **enterprise-level** applications easily and efficiently.

- Spring was introduced to simplify Java development by handling all those complex parts like object creation, dependency management, transactions, and security automatically.

Spring Architecture Diagram



Inversion of Control (IOC)

- Inversion of Control (IoC) is a principle in software engineering where the control of object creation and management is transferred (or inverted) from the developer to the framework

Why IOC ?

- ✓ **Loose Coupling** — classes don't depend directly on concrete implementations
- ✓ **Easier Testing** — objects can be replaced with mock dependencies
- ✓ **Better Code Management** — Spring handles the lifecycle of beans automatically

Two main container types

BeanFactory

ApplicationContext

BeanFactory

- BeanFactory is the simplest container in Spring that provides basic IoC (Inversion of Control) and Dependency Injection support.
- Creates beans **lazily** (only when requested)
- Lightweight
- Doesn't support advanced features like event publishing or internationalization

```
BeanFactory factory = new XmlBeanFactory(new ClassPathResource("applicationContext.xml"));
Student s = (Student) factory.getBean("s1");
s.display();
```

ApplicationContext

- ApplicationContext is a more advanced container that extends BeanFactory and provides additional enterprise-level functionality.

```
ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml");
Student s = context.getBean("s1");
s.display();
```

Steps To Create Spring Application

1. Create the Java Project.
2. Add spring jar files
3. Create Java class
4. Create the xml file
5. Create the Main method class

FortuneCloud
We Understand Technology

Steps

1. Create a new **Java Project**

2. Create bean/model file in your package like **Demo/Student.java**

```
package Demo;

public class Student {
    private String name;
    private String address;

    //generate getter & setters

    public void displayinfo() {
        System.out.println(name+" "+address);}
    }
}
```

3. Create **applicationContext.xml** file in **src** package

```
<?xml version="1.0" encoding="UTF-8"?>
<beans
      xmlns="http://www.springframework.org/schema/beans"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xmlns:p="http://www.springframework.org/schema/p"
      xsi:schemaLocation="http://www.springframework.org/schema/beans
      http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">

       <bean id="s1" class="Demo.Student">
              <property name="name" value="Abcd "></property>
              <property name="address" value="Pune "></property>
       </bean>

</beans>
```

4. Create **Test.java** with **main method**

```
ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml");

Student s = (Student) context.getBean("s1");
s.displayinfo();
```

OR

```
BeanFactory factory = new XmlBeanFactory(new ClassPathResource("applicationContext.xml"));
Student s = (Student) factory.getBean("s1");
s.display();
```

Dependency Injection (DI)

FortuneCloud
We Understand Technology

- Dependency Injection (DI) is a design pattern used by the Spring Framework to automatically provide (inject) the required objects (dependencies) into a class, instead of creating them manually.

- When **one class depends on another class**, that's a *dependency*

Types of Dependency Injection

1. Constructor Injection
2. Setter Injection

Setter Injection

- Setter Injection is a type of Dependency Injection where the Spring container injects dependencies using setter methods of a bean (class).

- Spring calls the setter methods after creating the bean to supply the required dependencies.

Student.java

```java
private String name;
private String address;
```

```java
public String getName() {
        return name;
}

public void setName(String name) {
        this.name = name;
}

public String getAddress() {
        return address;
}
public void setAddress(String address) {
        this.address = address;
}
```

Xml file

```
<bean id="s1" class="Demo.Student">
        <property name="name" value="abcd"></property>
        <property name="address" value="pune"></property>
</bean>
```

Constructor Injection

- Constructor Injection is a type of Dependency Injection (DI) where the Spring container injects dependencies through a class constructor.

- When Spring creates the object (bean), it passes the required dependencies as constructor parameters.

Student.java

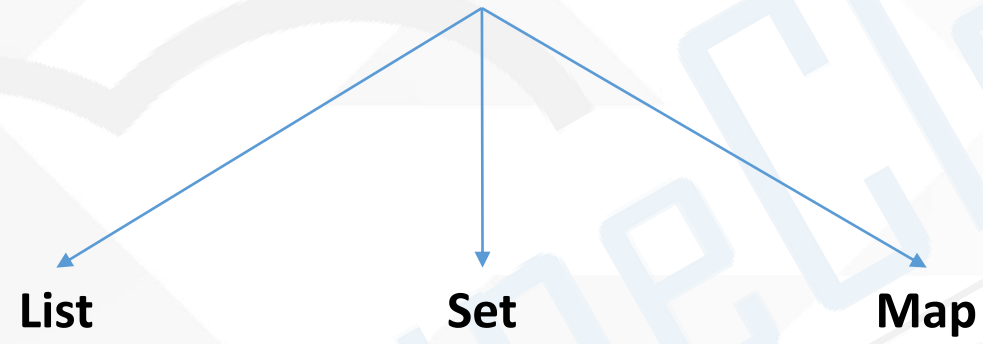
```
private int id;
private String name;
```

```
public Student() {System.out.println("Default Constructor");}

//single parameter constructor
public Student(int id) {this.id=id;}
public Student(String name) {this.name=name;}

public Student(int id, String name) {
        this.id=id;
        this.name=name;
}
```

Xml file

```
<bean id="e" class="Demo.Student">
        <constructor-arg value="10" type="int"></constructor-arg>
        <constructor-arg value="abc"></constructor-arg>
</bean>
```

Constructor Injection with Collection



Constructor Injection with Collection - List

FortuneCloud
We Understand Technology

Student.java //step 1

```
public List<String> skills;
```

Student.java //step 2

```
System.out.println(skills);
```

OR

```
Iterator<String> itr = skills.iterator();

while(itr.hasNext()) {
        System.out.println(itr.next());
}
```

Xml file

```
<constructor-arg>
        <list>
                <value>Java</value>
                <value>Python</value>
        </list>
</constructor-arg>
```

Constructor Injection with Collection - Set

FortuneCloud
We Understand Technology

Student.java //step 1

```
private Set<String> departments;
```

Student.java //step 2

```
System.out.println(departments);
```

OR

```
Iterator<String> itr = departments.iterator();

while(itr.hasNext()) {
        System.out.println(itr.next());
}
```

Xml file

```
<constructor-arg>
        <set>
                <value>Computer</value>
                <value>Mechanical</value>
        </set>
</constructor-arg>
```

Constructor Injection with Collection - Map

Xml file

Student.java //step 1

```java
private Map<String, String> books;
```

```xml
<constructor-arg>
    <map>
        <entry key="Book 1" value="Author 1"/>
        <entry key="Book 2" value="Author 2"/>
    </map>
</constructor-arg>
```

Student.java //step 2

```java
System.out.println(books);
```

OR

```java
Iterator<Map.Entry<String, String>> itr2 =
books.entrySet().iterator();

while (itr2.hasNext()) {
  Map.Entry<String, String> e = itr2.next();
  System.out.println(e.getKey() + " " + e.getValue());
}
```